

Why FPGAs Beat CPUs for Tick-to-Trade

Parallelism, DSP Wiring, and the Multi-Core Question

FPGA-Accelerated HFT Project

2026-06-12

The core idea. On a CPU, “steps” happen one after another *in time* on a shared engine. On an FPGA, “steps” are physical circuits laid out *in space*, and they all run *every clock cycle at once*. For a short, fixed, latency-critical pipeline like tick-to-trade, that difference is decisive — and, crucially, deterministic.

1 Two Flavours of FPGA Parallelism

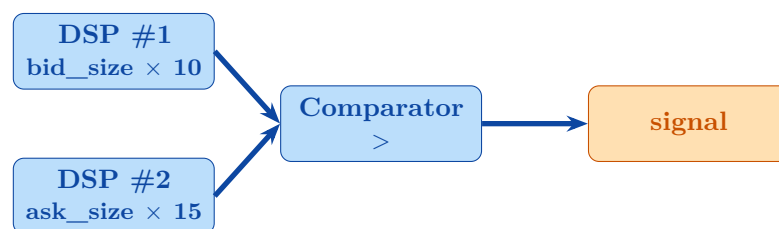
Before comparing against CPUs, it is worth separating the two distinct things people mean by “the FPGA runs in parallel.”

1. Spatial parallelism (within one cycle)

Consider the imbalance comparison, expressed without division via the cross-multiplication identity $a/b > k \iff a \cdot k_d > k_n \cdot b$:

$$\text{bid_size} \times 10 > 15 \times \text{ask_size}.$$

On a CPU this is two multiplies and a compare — three sequential instructions sharing one ALU. On the FPGA there are **two physical DSP multiplier blocks sitting side by side on the die**. One computes $\text{bid_size} \times 10$, the other computes $15 \times \text{ask_size}$, *at the same time*, because they are different lumps of silicon. Their outputs feed a comparator. Everything resolves within a **single clock cycle**.



all in ONE cycle — the two multiplies happen simultaneously

2. Pipeline parallelism (across cycles)

Each stage of the tick-to-trade path is its own circuit with a register at its output. On **every clock cycle**, all stages are active — each working on a *different message*.

cycle →	1	2	3	4	5
Stage A: parse type	msg1	msg2	msg3	msg4	msg5
Stage B: shift fields	—	msg1	msg2	msg3	msg4
Stage C: update book	—	—	msg1	msg2	msg3
Stage D: compute signal	—	—	—	msg1	msg2

By cycle 4 the pipeline is full: Stage A parses msg4 while Stage D computes the signal for msg1 — *all four in the same cycle*. A CPU has one engine, so it can only be “in” one stage at a time.

2 What “DSP Wiring” Means

DSP = Digital Signal Processing block — a small, hardened, pre-built multiply-accumulate circuit baked into the FPGA silicon. On the Xilinx VU9P (the AWS F1 chip) these are **DSP48E2 slices**, and there are about **6,840** of them scattered across the die. Each can compute, in one cycle:

$$\text{result} = (A + D) \times B + C,$$

a multiplier feeding an adder/accumulator with optional I/O registers. It is not “programmed” like a CPU instruction — it is a physical block of optimised transistors that does multiply-add and nothing else.

Wiring it means: in your RTL, when you write

```
1 wire [47:0] left  = bid_size * 16'd10;    // synthesizes to DSP #1
2 wire [47:0] right = ask_size * 16'd15;    // synthesizes to DSP #2
3 wire signal = (left > right);             // synthesizes to LUT
    comparator
```

the synthesis tool **maps** each multiply onto a physical DSP block. The “wires” are the actual metal routing carrying `bid_size`’s bits *into* DSP #1’s input port and its 48-bit output *out* toward the comparator. You are literally describing which silicon blocks connect to which.

Temporal vs. spatial. On a CPU, `bid_size * 10` is a *temporal* event — an instruction that occupies the one ALU for some cycles. On the FPGA it is a *spatial* fact — a specific block at specific die coordinates, always there, always wired the same way.

3 “But CPUs Have Many Cores” — The Three Reasons

Multi-core is real parallelism, but it is a **coarse, expensive** kind that does not help the *small* operations on a tick-to-trade path. Three reasons.

Reason 1 — Core count vs. operation count

	CPU core	FPGA LUT / DSP
Count	32–64	~1.2M LUTs / ~6,840 DSPs
Does	<i>any</i> instruction, one at a time	<i>one</i> fixed tiny op, always
Parallel unit	a whole thread	a single gate / multiply

A CPU core is a heavyweight generalist. To run your two multiplies “in parallel” on two cores, each core still fetches, decodes, and executes — hundreds of cycles of overhead to parallelize an operation the FPGA does in *one* cycle with zero overhead.

Reason 2 — The unit of parallelism is too big

Cores parallelize at the granularity of **threads** — chunks of work big enough to justify the cost of coordination. Splitting `bid_size × 10` and `ask_size × 15` across two cores is absurd: the work is nanoseconds, but launching/syncing threads costs **microseconds**. You would lose by orders of magnitude. FPGA parallelism is at the granularity of **a single arithmetic operation, or even a single bit** — no thread, no scheduler, no coordination cost. That is exactly the scale this problem lives at.

Reason 3 — Cores still share memory and are not deterministic

Even if you did spread work across cores, they share:

- **The cache hierarchy** — one core's miss stalls it $\sim 100+$ cycles, unpredictably.
- **The memory bus** — contention between cores adds jitter.
- **Coherency traffic** — keeping caches in sync between cores costs cycles, non-deterministically.
- **The OS scheduler** — it can migrate or preempt your thread at any moment.

Multi-core gives more *throughput* for big independent jobs (rendering, compiling, serving many requests), but does **nothing** for the *latency* of one small dependent chain like `parse` → `update` → `compare`.

Side by side

**CPU (even
64 cores)**

realistically runs on
one core, sequentially:
[mul][mul][cmp] + risk
of cache miss / preempt.
Splitting across cores costs μ s to save ns.

FPGA

DSP#1: `bid × 10` | DSP#2: `ask × 15`
→ comparator → signal,
all in ONE cycle, every cycle, forever.

4 The Real Distinction

It is not “FPGA has more parallelism than a multi-core CPU.” It is that they parallelize at **different granularities**:

- **CPU cores:** a *few* generalist engines, parallel at the **thread** level. Great for many big independent tasks. Overhead too high for tiny ops; shared resources kill determinism.
- **FPGA:** *millions* of tiny specialist circuits, parallel at the **operation/bit** level. Each does one fixed thing every cycle. Perfect for a short, fixed, latency-critical pipeline.

For HFT the bottleneck is **the latency of one message through one short chain**, not the throughput of millions of independent jobs. Multi-core attacks the wrong axis; the FPGA attacks exactly yours — and does it deterministically:

$$p_{99} = p_{50} = p_1.$$

Bottom line. The slowest message and the median message take *identically* long. No cache to miss, no branch to mispredict, no OS to interrupt. For tick-to-trade, predictable worst-case latency is worth more than raw average speed — and that is something cores fundamentally cannot promise.